

Partie 2 : Exercices & Architecture (Format Jupyter Notebook)

Configuration et Prérequis (Set Up pour Debian/Ubuntu)

L'environnement de travail repose sur un serveur local et l'utilisation conjointe de requêtes HTTP, de l'interface en ligne de commande (CLI) et du SDK Python officiel `hvac`.

1. Installation système et démarrage

Cette étape garantit que le logiciel téléchargé provient bien de l'éditeur et n'a pas été altéré.

```
```Bash
1. Ajout de la clé GPG
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg

2. Ajout du dépôt officiel
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list

3. Installation du binaire natif
sudo apt update && sudo apt install vault

4. Démarrage de l'infrastructure de développement (Garder ce terminal ouvert)
vault server -dev

5. Installation des dépendances Python (Dans un nouveau terminal)
uv add hvac requests
```
```

2. Initialisation de l'environnement Python

L'architecture de configuration accepte l'injection de paramètres par ordre de priorité croissante via des fichiers, des drapeaux CLI, et des variables d'environnement. Les scripts liront ces variables (`VAULT_ADDR`, `VAULT_TOKEN`).

```
```Python
import os
import hvac

Définition éphémère pour le serveur local
os.environ["VAULT_ADDR"] = "http://127.0.0.1:8200"
os.environ["VAULT_TOKEN"] = "hvs.MON_TOKEN_EPHEMERE_LOCAL" # À remplacer par le Root
Token du terminal

Initialisation du client HVAC
client = hvac.Client(url=os.getenv("VAULT_ADDR"), token=os.getenv("VAULT_TOKEN"))
leader_status = client.sys.read_leader_status()
print(f"Connexion réussie. Le serveur est prêt (Leader HA: {leader_status['ha_enabled']})")
```
```

⚠ **Note d'Architecture (Sécurité)** Déployer des jetons Vault en dur dans le code d'une application est une faille de sécurité majeure. En production, le code doit rester agnostique et consommer les variables d'environnement injectées par l'infrastructure. L'injection artificielle via `os.environ` ci-dessus est une facilité exclusive au mode de développement.

Niveau 1 : Fondamentaux du Moteur Key/Value (KV-V2) et Bootstrapping

L'objectif de ce niveau est de familiariser les développeurs avec le cycle de vie des secrets statiques. Le KV-V2 offre une protection native contre l'écrasement de données concurrentes via le mécanisme Check-And-Set (CAS).

1. Activation et mécanisme CAS

Le paramètre `cas_required=True` impose à tout client de prouver qu'il connaît la **version actuelle du secret** avant de le modifier. Si la version ne correspond pas, l'API lèvera une exception `hvac.exceptions.InvalidRequest`.

```
'''Python
import hvac

try:
    # On s'assure que le moteur est activé sur le chemin par défaut 'secret'
    client.sys.enable_secrets_engine(backend_type="kv", path="secret", options={"version": "2"})
    print("Moteur KV-v2 activé sur le chemin 'secret/'")
except hvac.exceptions.InvalidRequest as e:
    if "path is already in use" in str(e):
        print("Le moteur est déjà activé sur 'secret/'. Poursuite...")
    else:
        raise

# Configuration stricte de l'historique et du CAS sur ce point de montage
client.secrets.kv.v2.configure(max_versions=20, cas_required=True, mount_point="secret")
'''
```

2. Écriture, Extraction et Sous-clés

L'architecture sépare conceptuellement les données du secret de **ses métadonnées** et de **sa structure**. Lors de l'exploration via l'endpoint `/subkeys/`, Vault remplace la valeur sous-jacente des clés par `null` (ou `nil`) afin de ne pas exposer les données en clair ni déclencher de fausses alertes dans les journaux d'audit.

```
'''Python
import requests

# Création du secret statique (cas=0 fixe la création initiale)
# C'est CE secret précis qui sera requis par l'Agent au Niveau 4
client.secrets.kv.v2.create_or_update_secret(
    mount_point="secret",
    path="database/config",
    secret={"password": "SuperSecretDB2026!"},
    cas=0,
)
```

```
# Lecture des métadonnées complètes
secret_resp = client.secrets.kv.v2.read_secret_version(mount_point="secret",
path="database/config")
print(f"Version actuelle : {secret_resp['data']['metadata']['version']}")

# Exploration des sous-clés sans exposer la valeur du mot de passe
subkeys_endpoint = f"{os.getenv('VAULT_ADDR')}/v1/secret/subkeys/database/config"
subkeys_resp = requests.get(subkeys_endpoint, headers={"X-Vault-Token":
os.getenv("VAULT_TOKEN")})
print("Structure :", subkeys_resp.json()["data"]["subkeys"])
...

```

3. Politique Centralisée de Mots de Passe

Il est possible d'implémenter une politique centralisée pour imposer des chaînes aléatoires conformes aux développeurs, via l'API [sys/policies/password/nom](#).

```
'''Python
password_policy = """
length = 20
rule "charset" { charset = "abcdefghijklmnopqrstuvwxyz" min-chars = 2 }
rule "charset" { charset = "ABCDEFGHIJKLMNOPQRSTUVWXYZ" min-chars = 2 }
rule "charset" { charset = "0123456789" min-chars = 2 }
rule "charset" { charset = "!@#$%^&*" min-chars = 2 }
"""

policy_endpoint = f"{os.getenv('VAULT_ADDR')}/v1/sys/policies/password/norme-entreprise"
requests.post(policy_endpoint, headers={"X-Vault-Token": os.getenv("VAULT_TOKEN")},
json={"policy": password_policy})
...
'''

```

Niveau 2 : Automatisation M2M avec AppRole

AppRole est la méthode d'authentification privilégiée pour les architectures "Machine-to-Machine" reposant sur un **RoleID** (comparable à un login public) et un **SecretID** (mot de passe éphémère). Le flux sécurisé exige un mécanisme de "Pull", où l'application tire son identifiant depuis une source sécurisée, contrairement au mode "Push" qui est déconseillé.

1. Définition des Contraintes de l'AppRole

La création d'un rôle offre un contrôle granulaire des **politiques d'accès** et des **restrictions réseau**.

```
'''Python
try:
    client.sys.enable_auth_method(method_type="approle", path="approle")
except hvac.exceptions.InvalidRequest:
    pass

client.auth.approle.create_or_update_approle(
    role_name="backend-microservice",
    bind_secret_id=True, # Force la présentation du SecretID lors du login
    local_secret_ids=False,
    secret_id_num_uses=1, # Le SecretID ne peut être utilisé qu'une seule fois

```

```
secret_id_ttl="10m", # Le SecretID p rime 10 minutes apr s sa cr ation
secret_id_bound_cids=["127.0.0.1/32"], # Restreint la demande au r seau interne
token_policies=["default"],
)
...
```

2. Authentification Autonome

Lors de la g n ration, l'administrateur peut attacher des m tadonn es au format JSON pour assurer un suivi op rationnel dans les **journaux d'audit**.

```
'''Python
# Extraction du RoleID (Syst me de d ploiement)
role_id = client.auth.approle.read_role_id(role_name="backend-microservice")["data"]["role_id"]

# G n ration dynamique du SecretID avec attachement de m tadonn es
secret_id = client.auth.approle.generate_secret_id(
    role_name="backend-microservice",
    metadata={"env": "production", "region": "eu-west"}
)["data"]["secret_id"]

# Consommation par l'application (Nouveau client vierge)
app_client = hvac.Client(url=os.getenv("VAULT_ADDR"))
app_client.auth.approle.login(role_id=role_id, secret_id=secret_id)
print(f"Application authentifi e : {app_client.is_authenticated()}")
'''
```

Niveau 3 : Cryptographie (Transit) et Response Wrapping

Le moteur **Transit**, ou "**Encryption as a Service**", soulage les d veloppeurs de la complexit  cryptographique. Une exigence stricte de ce moteur est que toutes les donn es en clair transitant par son API doivent  tre encod es en base64 pour s curiser la transmission sans corrompre les structures JSON.

1. Encodage et Chiffrement

```
'''Python
import base64

try:
    client.sys.enable_secrets_engine(backend_type="transit", path="transit")
except hvac.exceptions.InvalidRequest:
    pass

# Sp cification de l'algorithme cryptographique (aes256-gcm96 par d faut)
client.secrets.transit.create_key(name="payment-key", key_type="aes256-gcm96")

# Chiffrement (L'encodage B64 est un pr requis technique, pas une s curit )
plaintext = "4000-1234-5678-9010"
encoded_text = base64.b64encode(plaintext.encode("utf-8")).decode("utf-8")

encrypt_resp = client.secrets.transit.encrypt_data(name="payment-key", plaintext=encoded_text)
```

```

ciphertext = encrypt_resp["data"]["ciphertext"]
print(f"Texte chiffré généré par Vault : {ciphertext}")

# Déchiffrement local
decrypt_resp = client.secrets.transit.decrypt_data(name="payment-key", ciphertext=ciphertext)
decrypted_text = base64.b64decode(decrypt_resp["data"]["plaintext"]).decode("utf-8")
print(f"Donnée récupérée en clair : {decrypted_text}")

```

 **Note d'Architecture : Envelope Encryption** Pour de très hauts volumes, le modèle d'Envelope Encryption génère une Data Encryption Key (DEK) en clair pour chiffrer en mémoire, et une Encrypted Data Key (EDK) à stocker durablement, évitant ainsi le goulet d'étranglement réseau.

2. Response Wrapping (Transmission Sécurisée)

Le "Response Wrapping" insère une donnée dans un **stockage éphémère (Cubbyhole)** et retourne un jeton d'emballage jetable. Si une tentative de déballage échoue car le jeton a déjà été utilisé, cela prouve mathématiquement son interception et déclenche une investigation.

```

'''Python
# 1. Enveloppement de la donnée (Action de l'émetteur)
wrap_resp = client.sys.wrap(payload={"mot_de_passe_admin": "SuperSecret123!"}, ttl="60s")
wrapping_token = wrap_resp["wrap_info"]["token"]

# 2. Déballage de la donnée (Action du destinataire)
unwrap_client = hvac.Client(url=os.getenv("VAULT_ADDR"), token=wrapping_token)
unwrap_resp = unwrap_client.sys.unwrap()
print(f"Secret déballé avec succès : {unwrap_resp['data']}")

# 3. Test de sécurité (Simulation d'une attaque par rejeu)
try:
    unwrap_client.sys.unwrap()
except hvac.exceptions.InvalidRequest as e:
    print(f"Sécurité confirmée. Erreur de l'API suite à la double utilisation : {e}")
'''

```

Niveau 4 : Expert - Automatisation Déclarative via Vault Agent

Lorsque le code applicatif ne peut pas intégrer le SDK HVAC, **Vault Agent** prend le relais. Le démon s'authentifie de manière autonome (**Auto-Auth**) et gère le cycle de vie des certificats via le moteur de **Templating**, réduisant la complexité applicative tout en garantissant les plus hautes normes de sécurité.

1. Configuration de l'Autorité de Certification (PKI)

L'équipe sécurité active le moteur PKI, génère une **autorité de certification (CA)** et configure un **rôle** qui autorise la génération de certificats avec une durée de vie maximale très courte.

```

'''Python
try:
    client.sys.enable_secrets_engine(backend_type="pki", path="pki")
    client.sys.tune_mount_configuration(path="pki", max_lease_ttl="87600h")
'''

```

```

except hvac.exceptions.InvalidRequest:
    pass

try:
    client.secrets.pki.generate_root(type="internal", common_name="app-internal-ca",
    extra_params={"ttl": "87600h"})
except Exception:
    pass

client.secrets.pki.create_or_update_role(
    name="role-serveur-web",
    extra_params={"allowed_domains": "mon-app.local", "allow_subdomains": True, "max_ttl": "1h",
    "key_type": "rsa", "key_bits": 2048}
)

```

2. Déploiement Idempotent de la Sécurité et de l'Agent

L'Agent nécessite l'attachement d'une **Politique (Policy)** explicite liant son identité (**Authentification**) à ses habilitations (**Autorisation**). Ce script garantit une reconstruction complète et saine de la chaîne de permissions.

```

'''Python
import os

# 1. Autorisation : Création de la Politique liant KV-v2 et PKI
policy_hcl = """
path "secret/data/database/config" { capabilities = ["read"] }
path "pki/issue/role-serveur-web" { capabilities = ["create", "update"] }
"""

client.sys.create_or_update_policy(name="agent-policy", policy=policy_hcl)

# 2. Authentification : Création de l'AppRole et écriture des identifiants
client.auth.approle.create_or_update_approle(role_name="backend-microservice",
token_policies=["agent-policy"])
role_id = client.auth.approle.read_role_id(role_name="backend-microservice")["data"]["role_id"]
secret_id = client.auth.approle.generate_secret_id(role_name="backend-microservice")["data"]["secret_id"]

with open("role_id.txt", "w") as f: f.write(role_id)
with open("secret_id.txt", "w") as f: f.write(secret_id)

# 3. Templating : Modèle hybride (Statique KV + Dynamique PKI)
template_content = """
[DATABASE]
{{ with secret "secret/data/database/config" }}Mot de passe = {{ .Data.data.password }}{{ end }}

[TLS]
{{ with secret "pki/issue/role-serveur-web" "common_name=api.mon-app.local" "ttl=5m" }}Certificat =
{{ .Data.certificate }}{{ end }}
"""

with open("app_config.tpl", "w") as f: f.write(template_content)

# 4. Configuration finale du Démon
hcl_content = """
vault { address = "http://127.0.0.1:8200" }

```

```

auto_auth {
  method "approle" {
    config = {
      role_id_file_path = "role_id.txt"
      secret_id_file_path = "secret_id.txt"
      remove_secret_id_file_after_reading = false
    }
  }
}
template { source = "app_config.tpl" destination = "config_genereee.txt" }
"""
with open("agent.hcl", "w") as f: f.write(hcl_content)
"""

```

💡 **Note d'Architecture** : Le fichier modèle `app_config.tpl` agrège deux données distinctes : le certificat dynamique du PKI et le mot de passe statique du moteur KV. Si le secret statique venait à manquer, l'Agent bloquerait la génération du fichier pour éviter de crasher l'application. Puisque nous avons correctement injecté `database/config` lors de la phase de Bootstrapping au **Niveau 1**, l'Agent exécutera son rendu sans la moindre erreur.

3. Exécution Finale

La configuration fonctionnelle peut être testée directement en lançant le démon dans le terminal système, au sein du dossier contenant les fichiers générés.

```

'''Bash
vault agent -config=agent.hcl
'''

```

FIN.
 319. 337★
 ^ (GOOD JOB!)

Sources

Tutoriels Officiels & Guides de Prise en Main (Exercices Pratiques)

- Bibliothèque globale des tutoriels HashiCorp Vault : <https://developer.hashicorp.com/vault/tutorials>
- Parcours d'initiation "Vault Foundations" : <https://developer.hashicorp.com/vault/tutorials/get-started>
- Operations Quick Start (CLI, KV, AppRole, Policies) : <https://developer.hashicorp.com/vault/docs/get-started/operations-qs>
- Prise en main de l'interface en ligne de commande (CLI) : <https://developer.hashicorp.com/vault/tutorials/get-started/learn-cli>
- Prise en main de l'interface web (UI) : <https://developer.hashicorp.com/vault/tutorials/get-started/learn-ui>

SDK Python pour l'intégration dans Jupyter Notebook

Pour exécuter du code Python interactif au sein de vos notebooks Jupyter, l'usage du SDK officiel `hvac` est recommandé :

- Dépôt GitHub du client Python `hvac` : <https://github.com/hvac/hvac>
- Documentation officielle du client `hvac` : <https://python-hvac.org/>